SIC AI Chapter Quiz: 06

Student Name: Devesh Panwar

Candidate ID: SIC202500060

Question 01

Option 4: K-means clustering has the advantage of being easy to determine the initial number of clusters and easy to interpret the results.

```
from sklearn.cluster import KMeans
import numpy as np

# Sample Data
X = np.array([[1, 2], [1, 4], [1, 0],
              [10, 2], [10, 4], [10, 0]])

# We have to GUESS or KNOW that there are 2 clusters here.
# If we don't set n_clusters, the code will default to 8 (which would be wrong for 6 data points).
kmeans = KMeans(n_clusters=2, random_state=0)

kmeans.fit(X)

print(f"Cluster Centers:\n{kmeans.cluster_centers_}")
print(f"Labels: {kmeans.labels_}")
```

```
Cluster Centers:
[[10.  2.]
 [ 1.  2.]]
Labels: [1 1 1 0 0 0]
```

Question 02

Option 3: The algorithm of hierarchical clustering has K-Medoids.

Question 03

Answer The EM (Expectation-Maximization) algorithm is considered a Greedy Algorithm because at every iteration, it chooses the parameters that maximize the current expected log-likelihood, ensuring the model improves (or stays the same) strictly step-by-step.

A "Greedy" algorithm is one that makes the locally optimal choice at each stage with the hope of finding a global optimum. The EM algorithm fits this description for the following reasons:

Monotonic Improvement: In the M-Step (Maximization step), the algorithm updates the parameters to strictly maximize the function found in the E-Step. This guarantees that the likelihood of the model never decreases; it always goes "uphill."

Local Optimization: Because it always greedily climbs the nearest slope of likelihood, it converges to a local maximum (a peak nearby) rather than searching the entire landscape for the global maximum (the highest peak overall). It does not "look ahead" or accept temporary bad steps to find a better long-term solution.

Question 04

Here are three common examples of clustering algorithms used in data analysis:

K-Means ClusteringHow it works: This is a partitioning method that splits data into k distinct groups. It works by assigning each data point to the nearest "centroid" (center of the cluster) and then recalculating the centroids based on the new assignments. It repeats this until the clusters stop changing.

- Best for: Large, evenly sized datasets with spherical clusters.Hierarchical

ClusteringHow it works: This method builds a tree-like hierarchy of clusters (a dendrogram). It can be Agglomerative (starting with single points and merging them bottom-up) or Divisive (starting with one big cluster and splitting it top-down). You can "cut" the tree at different levels to decide how many clusters you want.

- Best for: Smaller datasets where seeing the hierarchy/relationship between groups is important.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)How it works: Unlike K-Means, DBSCAN groups points that are closely packed together (high density). Points that lie alone in low-density regions are marked as outliers or noise. It does not require you to specify the number of clusters in advance.

- Best for: Data with irregular shapes and noise (outliers).

Question 5: Choosing Optimal Clusters

1. **The Elbow Method:**
    - Involves plotting the **Inertia (SSE)** against the number of clusters.
    - The optimal k is found at the "elbow" of the curve, where adding more clusters no longer significantly reduces the error.
2. **Silhouette Analysis:**
    - Measures how close each point is to points in its own cluster vs. points in neighboring clusters.
    - The optimal k is the one that maximizes the **Average Silhouette Score**.

Question 06

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN

# --- 1. Generate Data (Given in the question) ---
# Note: cluster_std=5 makes the blobs very spread out and overlapping
X, label = make_blobs(n_samples=300, n_features=2, centers=3, cluster_std=5, random_state=123)

# --- 2. Initialize the 3 Models ---

# A. K-Means (Requirement: n_clusters=3)
kmeans = KMeans(n_clusters=3, random_state=123, n_init=10)
kmeans_labels = kmeans.fit_predict(X)

# B. Agglomerative Clustering (Requirement: n_clusters=3)
agglo = AgglomerativeClustering(n_clusters=3)
agglo_labels = agglo.fit_predict(X)

# C. DBSCAN
# Note: DBSCAN does not use n_clusters.
# We set eps=2 because the data spread (std=5) is large. Default eps=0.5 would likely mark everything as r
dbscan = DBSCAN(eps=2, min_samples=5)
dbscan_labels = dbscan.fit_predict(X)

# --- 3. Visualization ---
plt.figure(figsize=(15, 10))

# Plot 1: Original Data
```

```
plt.subplot(2, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=label, cmap='viridis', alpha=0.7, edgecolor='k')
plt.title('1. Original Data')

# Plot 2: K-Means
plt.subplot(2, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=kmeans_labels, cmap='viridis', alpha=0.7, edgecolor='k')
plt.title('2. K-Means (k=3)')

# Plot 3: Agglomerative Clustering
plt.subplot(2, 2, 3)
plt.scatter(X[:, 0], X[:, 1], c=agglo_labels, cmap='viridis', alpha=0.7, edgecolor='k')
plt.title('3. Agglomerative Clustering (k=3)')

# Plot 4: DBSCAN
plt.subplot(2, 2, 4)
plt.scatter(X[:, 0], X[:, 1], c=dbscan_labels, cmap='viridis', alpha=0.7, edgecolor='k')
plt.title('4. DBSCAN')

plt.tight_layout()
plt.show()
```